

252-0027

Einführung in die Programmierung Übungen

Woche 3: Einführung Java

**Joran Bühler
Departement Informatik
ETH Zürich**

Organisatorisches

- Mein Name: Joran Bühler
- Bei Fragen: **buehlerjo@ethz.ch**
- Neue Aufgaben: **Dienstag Abend** (im Normalfall)
- Abgabe der Übungen bis **Dienstag Abend (23:59)** Folgewoche
 - Abgabe immer via Git
 - Lösungen in separatem Projekt auf Git
 - Korrektur (hoffentlich) bis Freitag

Ablauf

- Theory Recap
- Aufgaben zu Operatoren
- Kahoot
- (Input: Prüfungsaufgaben)
- Vorbesprechung

Auswertung – Arithmetische Operatoren

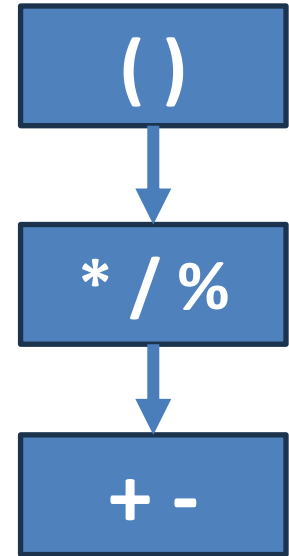
Operator	Beschreibung	Kurzbeispiel
+	Addition	<code>int</code> antwort = 40 + 2;
-	Subtraktion	<code>int</code> antwort = 48 - 6;
*	Multiplikation	<code>int</code> antwort = 2 * 21;
/	Division	<code>int</code> antwort = 84 / 2;
%	Teilerrest, Modulo-Operation, errechnet den Rest einer Division	<code>int</code> antwort = 99 % 57;
+	positives Vorzeichen	<code>int</code> j = +3;
-	negatives Vorzeichen	<code>int</code> minusJ = -j;

Assoziativität

- Die **Assoziativität** («**associativity**») eines Operators bestimmt, wie ein Operand verwendet wird
- Ist ein Operator $\otimes$
 - **Linksassoziativ** («**left-associative**») $\rightarrow X \otimes Y \otimes Z$ gleich $(X \otimes Y) \otimes Z$
 - **Rechtsassoziativ** («**right-associative**») $\rightarrow X \otimes Y \otimes Z$ gleich $X \otimes (Y \otimes Z)$

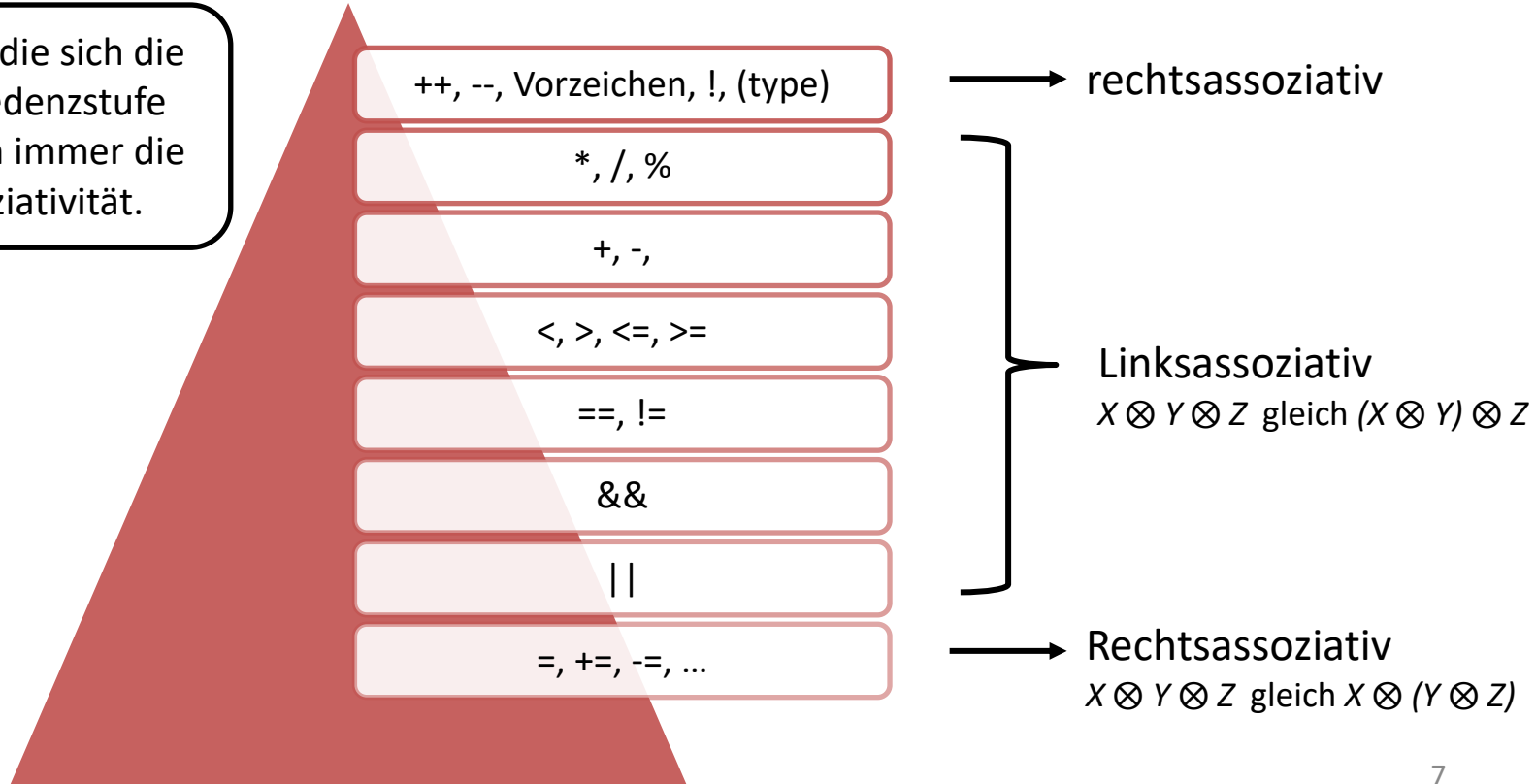
Auswertung – Ordnung

- Operand wird vom Operator mit höherer Rangordnung («precedence») verwendet
- Wenn zwei Operatoren dieselbe Rangordnung haben, dann entscheidet die Assoziativität
- Wenn etwas anderes gewünscht wird: Klammern verwenden!



Auswertung - detaillierter

Operatoren, die sich die gleiche Präzedenzstufe teilen, haben immer die gleiche Assoziativität.



Repetition: Casts



Kompatibilität ausgewählter Basistypen



		Zieltyp (Variable)			
Quelltyp (Ausdruck)		int	long	double	
	int				
	long				
	double				

explizite Konvertierung

implizite Konvertierung

implizite Konvertierung – potenziell mit Genauigkeitsverlust

<- Explizit

byte -> short -> int -> long -> float -> double

Implizit ->

Zeichenverkettung (ähnlich – aber nicht dasselbe!)

- Verkettung eines Strings mit einem Wert anderen Typs:
Verkettung mit der Standarddarstellung des anderen Werts
 - `3 + " Grad Celsius"` ergibt `"3 Grad Celsius"`
 - `"Olympia " + 2024` ergibt `"Olympia 2024"`
- Praktisch für die Erzeugung von Ausgabe

```
double grade = 5.25 + 0.25;  
System.out.println("Endnote ist " + grade);  
// erzeugt Ausgabe "Endnote ist 5.5"
```

Auswertung – Beispiele

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(2/3+4.5/3);  
4     }  
5 }
```

Output: 1.5

Auswertung – Beispiele

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(8%3+2.5+4/3);  
4     }  
5 }
```

Output: 5.5

Auswertung – Casting

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println((double)5/2);  
4     }  
5 }
```

Output: 2.5

Auswertung – String

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println("Wir schreiben das Jahr "+2023/2*2.0);  
4     }  
5 }  
6
```

Output: Wir schreiben das Jahr 2022.0

Tutorial: Alte Prüfungen

- Alte Prüfungen auf: <https://exams.vis.ethz.ch>
- Alte Prüfungen lösen ist nicht Teil des Semesters.
 - Dafür bleibt genügend Zeit in der Lernphase in der vorlesungsfreien Zeit vor den Prüfungen.
- Alte Prüfungsaufgaben werden von Zeit zu Zeit in den Übungen kommen, aber es verbleiben genügend alte Prüfungen, welche ihr selbstständig lösen könnt.

Kurzschlussauswertung («short-circuit evaluation»)

Bei Berechnung von `p && q` und `p || q`

- Java wertet zuerst den linken Operanden (`p`) und dann den rechten (`q`) aus
- die Auswertung wird beendet, sobald das Ergebnis feststeht.
 - Bei `p && q`: Falls `p == false`, dann ist `p && q == false`
 - Bei `p || q`: Falls `p == true`, dann ist `p || q == true`

p	q	p && q
true	true	true
true	false	false
false	true	false
false	false	false

p	q	p q
true	true	true
true	false	true
false	true	true
false	false	false

In diesen Fällen ist keine Auswertung von `q` nötig!

Kahoot

<https://create.kahoot.it/share/u3-casts-strings-and-arithmetic/5f5575b1-1269-4c34-b922-d722de1f8ab1>

Aufgabe 1

```
1  class MyClass{  
2      public static void main(String[] args){  
3          System.out.println(9 / 3 * 2 + 1 + "" + 2 * 3 + (4 * 3) % 3);  
4      }  
5  }
```

760

Aufgabe 2



```
1  class MyClass{
2      public static void main(String[] args){
3          System.out.println(8 / 5 + 0.5 + 5 / 2 + (8 % 3) * 1.0);
4      }
5  }
```

5.5

Nachbesprechungen der Übungen?

- Feedback und Lösungen in Github Repos

Vorbesprechung Übung 3

Aufgabe 1: Fehlerbehebung

Finden und beheben Sie alle Fehler im Programm “FollerVehler.java”. Eclipse hilft Ihnen dabei, indem es anzeigt, wo die Fehler sind (und eine mehr oder weniger hilfreiche Fehlermeldung dazu ausgibt), aber Sie müssen selber herausfinden, was das Problem ist und wie Sie es beheben können. Wenn Sie alle Fehler behoben haben, sollte das Programm folgendes ausgeben:

```
Hello world
```

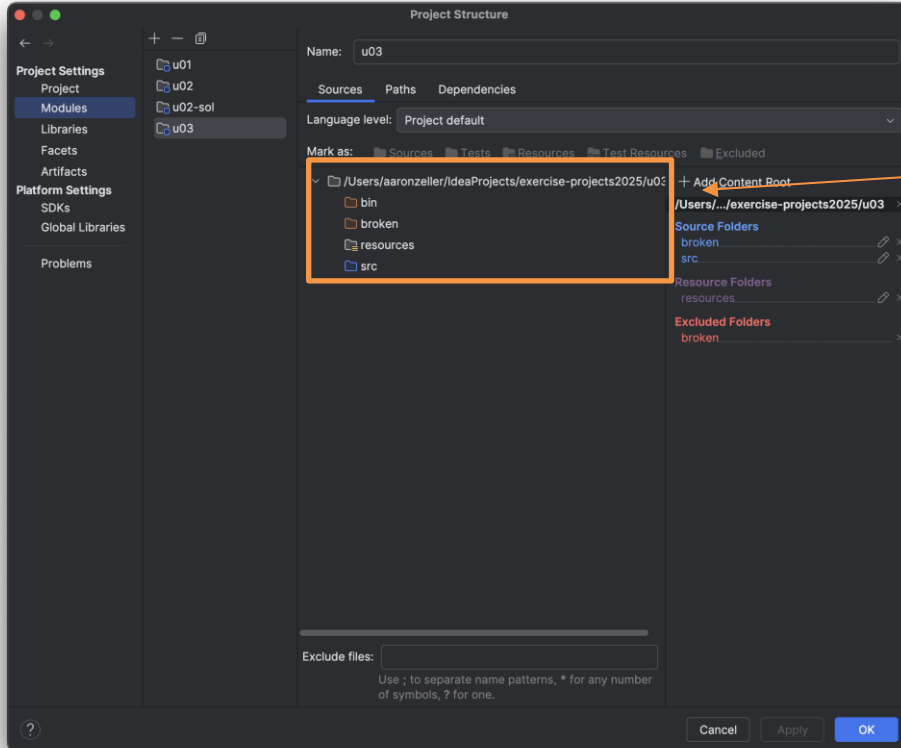
```
Gefällt Ihnen dieses Programm?
```

```
Ich habe es selbst geschrieben.
```

Aufgabe 1: Fehlerbehebung

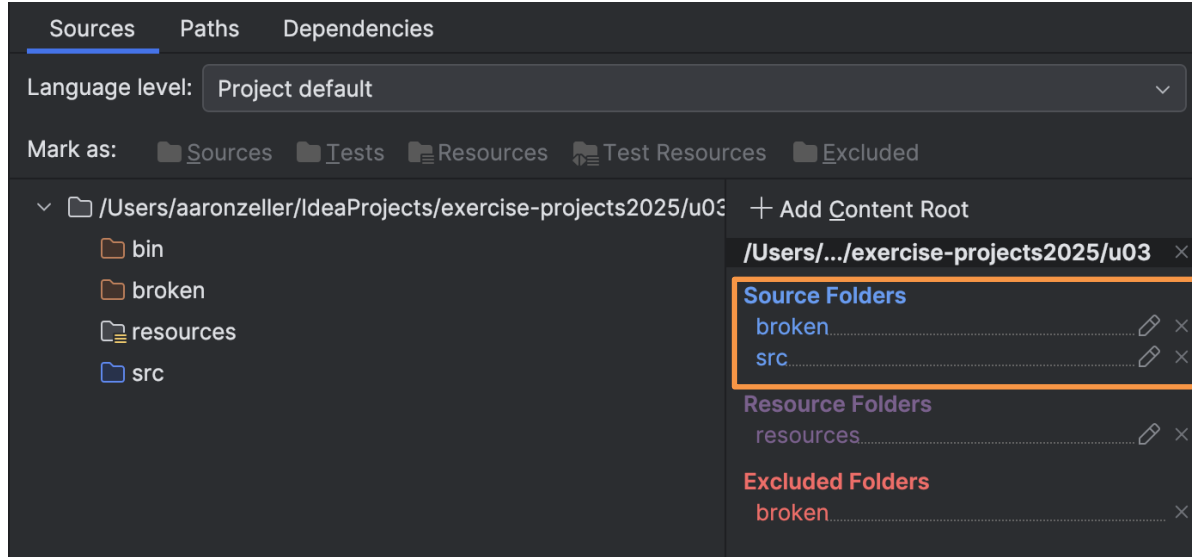
- In Java werden **alle** Java-Dateien in einem Projekt kompiliert.
- Wenn auch **nur eine** Java-Datei nicht kompiliert, dann kann **kein** Java-Code ausgeführt werden in dem Projekt.

Aufgabe 1: Fehlerbehebung



Hier sind die Unterordner

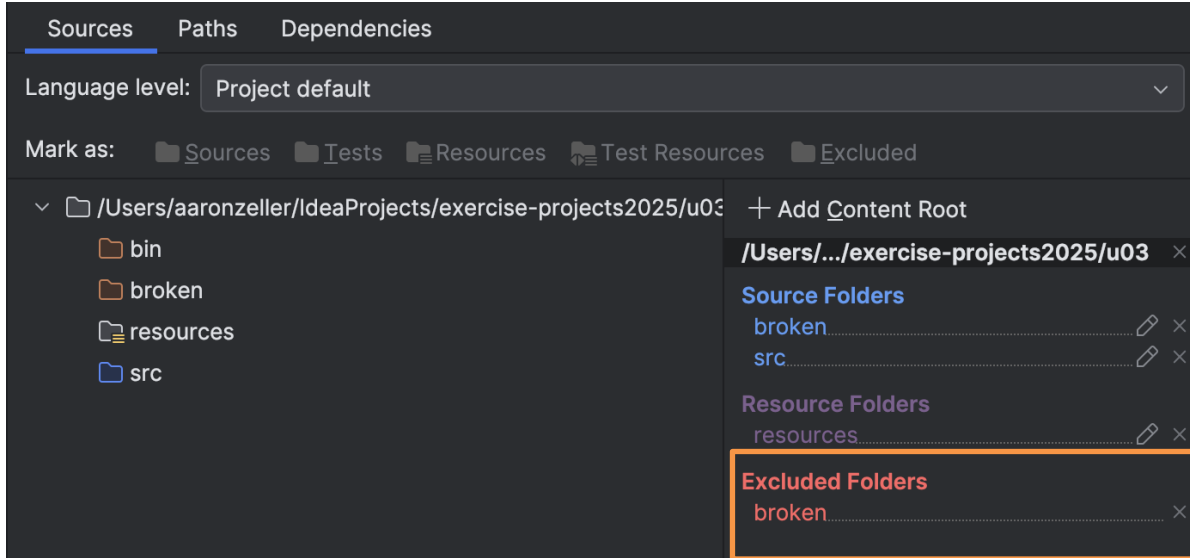
Aufgabe 1: Fehlerbehebung



Alle "Sources" werden kompiliert



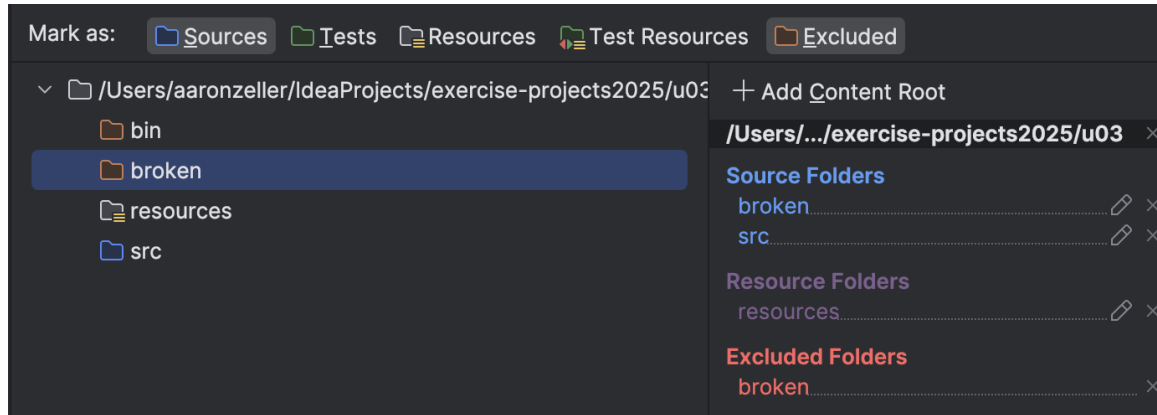
Aufgabe 1: Fehlerbehebung



Ausser sie sind "Excluded"

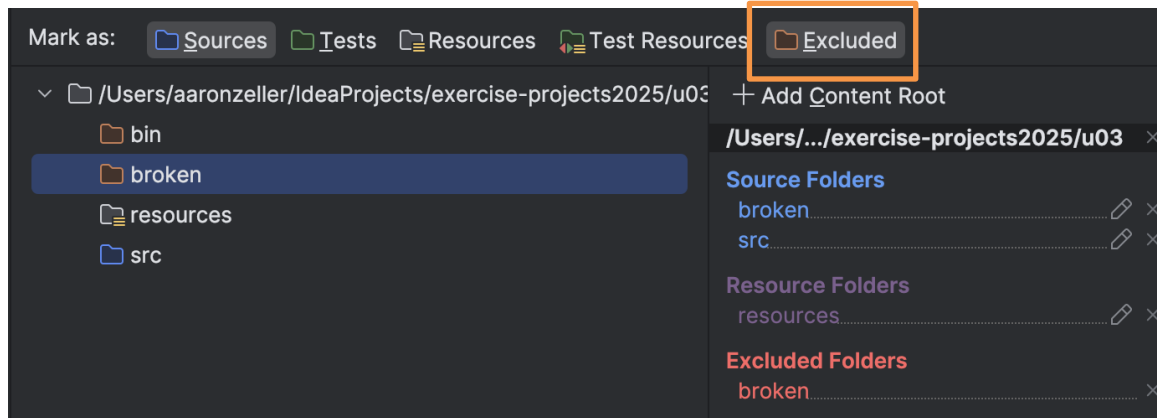


Aufgabe 1: Fehlerbehebung



Wenn man auf einen Unterordner klickt, dann kann man ihn als “Sources”, “Excluded”, “Resources”, etc. markieren.

Aufgabe 1: Fehlerbehebung



Wenn man auf “Excluded” klickt, dann wird der broken-Ordner wieder kompiliert!

Aufgabe 3: Eingabe und Zufall

Aufgabe 3: Eingabe und Zufall

In dieser Aufgabe arbeiten Sie mit der Eingabe und Ausgabe von Java und lernen die Klassen `Scanner` und `Random` näher kennen.

1. Schreiben Sie ein Programm "Adder.java", welches zwei ganze Zahlen einliest und die Summe davon ausgibt. Sie sollen dafür die `Scanner`-Klasse verwenden, wie in der Vorlesung gezeigt. Das Programm soll nach der ersten Zahl fragen:

Geben Sie Zahl 1 ein:

dann nach der zweiten Zahl:

Geben Sie Zahl 2 ein:

und schliesslich, wenn Sie zum Beispiel "4" und "1999" eingeben, folgendes ausgeben:

4 + 1999 = 2003

Sie können davon ausgehen, dass nur ganze Zahlen eingegeben werden. Testen Sie das Programm mit verschiedenen Zahlen.

2. Schreiben Sie ein Programm `Wuerfel.java`, das einen Würfel simuliert. Hierbei soll eine positive ganze Zahl `N` eingelesen werden, welche die Anzahl der Seiten des Würfels repräsentiert. Der übliche Würfel hat 6 Seiten, jedoch existieren auch Würfel mit 12, 16, 20 (siehe Abbildung 1) und mehr Seiten. Jede Seite trägt eine unterschiedliche Zahl, die von 1 bis `N` (inklusive `N`) reicht.

Das Programm soll den Wurf simulieren, indem es die Klasse `Random` verwendet. Ein möglicher Ablauf des Programmes könnte folgendermassen aussehen:

Wie viele Seiten hat Ihr Würfel?

Der Benutzer gibt eine Zahl ein, z.B. 20, danach wird gewürfelt:

Es wurde eine 17 gewürfelt!



3. Schreiben Sie ein Programm `ChatGTP.java`, welches die Interaktion mit einem AI-Chatbot simuliert. Das Programm soll den Benutzer begrüßen, Sie nach Ihrem Namen und Alter fragen, und dem Benutzer Ihre Glückszahl verraten. Die Glückszahl soll zufällig gewählt werden, in dem Sie die `Random` Klasse benutzen. Ein möglicher Ablauf des Programmes könnte folgendermassen aussehen:

Guten Tag! Ich bin ChatGTP, der beste Chatbot, denn es gibt! Wie heissen Sie?

Ein Namen wird eingegeben, z.B. Alice, danach antwortet der Chatbot auf diesen Input:

Sehr erfreut Alice! Wie alt sind Sie?

Ein Alter wird eingegeben, z.B. 23, danach antwortet der Chatbot auf diesen Input:

Mittels dieser Informationen habe ich Ihre Glückszahl gefunden! Die Glückszahl lautet 42.

Aufgabe 4: Berechnung

2. Vervollständigen Sie “SharedDigit.java”. In der Main-Methode sind zwei int Variablen a und b deklariert und mit einem Wert zwischen 10 und 99 (einschliesslich) initialisiert. Das Programm soll einer int Variablen r einen bestimmten Wert zuweisen. Wenn a und b eine Ziffer gemeinsam haben, dann wird r die gemeinsame Ziffer zugewiesen (wenn a und b beide Ziffern gemeinsam haben, dann kann eine beliebige Ziffer zugewiesen werden). Wenn es keine gemeinsame Ziffer gibt, dann soll -1 zugewiesen. Sie brauchen für dieses Programm keine Schleife.

Beispiele:

- Wenn a: 34 und b: 53, dann ist r: 3
- Wenn a: 10 und b: 22, dann ist r: -1
- Wenn a: 66 und b: 66, dann ist r: 6
- Wenn a: 34 und b: 34, dann ist r: 3 oder 4

Testen Sie Ihre Loesung mit a gleich 34 und b gleich 43. Was liefert Ihr Programm?

2. Vervollständigen Sie "SumPattern.java". In der Main-Methode sind drei int Variablen a, b, und c deklariert und mit irgendwelchen Werten initialisiert. Wenn die Summe von zwei der Variablen die dritte ergibt, nehmen wir an dass $a + c == b$, so soll die Methode "Moeglich. $a + c == b$ " ausgeben (wobei die Werte für a, b, und c einzusetzen sind). Wenn das nicht der Fall ist, dann soll die Methode "Unmoeglich." ausgeben.

Beispiele:

- Wenn a: 4, b: 10, c: 6, dann wird "Moeglich. $4 + 6 == 10$ " oder "Moeglich. $6 + 4 == 10$ " ausgegeben.
- Wenn a: 2, b: 12, c: 0, dann wird "Unmoeglich." ausgegeben.

3. Vervollständigen Sie "AbsoluteMax.java". In der Main-Methode sind drei int Variablen a, b, und c deklariert und mit irgendwelchen Werten initialisiert. Das Programm soll einer int Variable r den grössten absoluten Wert von a, b, und c zuweisen.

Aufgabe 5: EBNF

Aufgabe 5: EBNF

In dieser Aufgabe erstellen Sie erneut verschiedene EBNF-Beschreibungen. Speichern Sie diese wie gewohnt in der Text-Datei "EBNF.txt", welche sich in Ihrem "u02"-Ordner bzw. im "U02 <N-ETHZ-Account>"-Projekt befindet. Sie können die Datei direkt in Eclipse bearbeiten.

1. Erstellen Sie eine Beschreibung <pyramid>, welche als legale Symbole genau jene Wörter zulässt, welche aus einer Folge von strikt aufsteigenden, gefolgt von einer Folge von strikt absteigenden Ziffern bestehen. Beispiele sind: 14, 121, 1221, 1341.
Sie dürfen annehmen, dass das leere Wort auch zugelassen wird. (Als Challenge können Sie probieren, das leere Wort auszuschliessen.)

Aufgabe 5: EBNF

2. Erstellen Sie eine Beschreibung $\langle \text{digitsum} \rangle$, welche als legale Symbole genau jene natürlichen Zahlen zulässt, deren Quersumme eine gerade Zahl ist.
3. Erstellen Sie eine Beschreibung $\langle \text{xyz} \rangle$, welche genau alle Wörter zulässt, die aus X, Y und Z bestehen und bei welchen jedes X mindestens ein Y im Teilwort links und rechts von sich hat. Beispiele sind: Z, YXY, YXXY, ZYXYY.
4. Erstellen Sie eine Beschreibung $\langle \text{term} \rangle$, welche als legale Symbole genau alle wohlgeformten arithmetischen Terme, bestehend aus positiven ganzen Zahlen, Variablen (x, y, z), Addition und Klammern zulässt. Geklammerte Terme müssen mindestens eine Addition enthalten. Beispiele sind: $1 + 4$, $(1 + 4)$, $1 + (3 + 4)$, $(1 + 1) + x + 5$.